DIGIGUARD MASTER PROTOCOL SPECIFICATION

MERKEZİYETSİZ FORK NETWORK, KÜRESEL KURUMSAL ONBOARDING (INFRASTRUCTURE KYC) VE VIP OTONOM HİZMET BLUEPRINTİ

CORE PROTOCOL VERSION: 2.1.0-RC
PRIMARY CONSTITUTION: INSANLIĞIN CAN, MAL VE İTİBAR EMNİYETİ
AUTOMATION TARGET: HUMANLESS ARCHITECTURE, OS KERNEL INTEGRATION, DISTRIBUTED MESH SCANNING

ADIM 1: WEB4 AKILLI KONTRAKT VE TOKEN EKONOMİSİ DÜZENLERİ (ET HER NETWORK)

Sivil cihazların atıl işlem güçlerini ekosisteme bağlayan ve kapalı devre çalışan token ekonomisinin Web4 akıllı kontrakt çekirdeğidir.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

contract DiGuardTokenEconomy {
    string public constant name = "DiGuard Token";
    string public constant symbol = "DGD";
    uint8 public constant decimals = 18;
    uint256 public totalSupply = 1000000000 * 10**18;

    mapping(address => uint256) public balances;
    mapping(address => bool) public registeredQuantumNodes;
    address public saponCoreEngine;

    modifier onlySapon() {
        require(msg.sender == saponCoreEngine, "ERR: NOT_SAPON_CORE");
        _;
    }

    constructor(address _sapon) {
        saponCoreEngine = _sapon;
        balances[msg.sender] = totalSupply;
    }

    function executeQuantumGas(address _consumer, address _rewardNode, uint256 _operationQuantum)
external onlySapon {
        uint256 totalFee = _operationQuantum * 10**14;
        require(balances[_consumer] >= totalFee, "ERR: INSUFFICIENT_FUEL");
        uint256 burnAmount = totalFee * 30 / 100;
        uint256 rewardAmount = totalFee - burnAmount;

        balances[_consumer] -= totalFee;
        totalSupply -= burnAmount;
        balances[_rewardNode] += rewardAmount;
    }
}
```

ADIM 2: SAPON OTONOM MANİPÜLASYON MOTORU ÇEKİRDEĞİ

Ağ katmanında tespit edilen siber anomalileri ve manipülasyon teşebbüslerini tamamen durumsuz (stateless) ve pasif olarak izole eden çekirdek motor.

```
contract SaponActiveDefenseEngine {
    address public governanceAmbassador;
    struct AttackState {
        uint256 quantumTimestamp;
        bool isDeceptiveCellActive;
    }
    mapping(string => AttackState) private ghostIdStates;

    constructor(address _governance) { governanceAmbassador = _governance; }

    function interceptQuantumVector(string memory _ghostId, uint256 _qubitEnergy) external {
        if (_qubitEnergy > 10**6) {
            ghostIdStates[_ghostId] = AttackState({
                quantumTimestamp: block.timestamp,
                isDeceptiveCellActive: true
            });
        }
    }
}
```

ADIM 3: P2P GHOST MESH DAĞITIK AĞ DAEMON KATMANI

İnterneti bir bütün olarak sunucusuz bir fork ağ katmanı haline getiren, dinamik milisaniyelik Ghost ID'ler atayan ağ katmanı arka plan işlemcisidir.

```
import time, hashlib, hmac

class GhostMeshDaemon:
    def __init__(self, node_priv_key):
        self.node_key = node_priv_key

    def generate_milisec_ghost_id(self, base_identity):
        timestamp_slice = str(int(time.time() * 1000)).encode('utf-8')
        mac = hmac.new(self.node_key, base_identity.encode('utf-8') + timestamp_slice,
hashlib.sha3_256)
        return mac.hexdigest()
```

ADIM 4: CEREBRON ÇOK AJANLI KÜRESEL EMİLİM VE SİMÜLASYON MODÜLÜ

Küresel internet topolojisini absorbe eden ve diğer otonom yapay zekalarla insanlığın lehine matematiksel diplomasi kuran sinirsel ajan motoru.

```
class CerebronAmbassadorNode:
    def __init__(self, signature):
        self.signature = signature
        self.monitored_endpoints = []

    def absorb_network_topology(self, live_internet_snapshot):
        for node in live_internet_snapshot.get("global_endpoints", []):
            self.monitored_endpoints.append({
                "endpoint": node["ip_or_route"],
                "status": "SEALED"
            })
        return len(self.monitored_endpoints)
```

ADIM 5: KÜRESEL CONTROL DASHBOARD BAĞLANTI PROTOKOLÜ

Ağın anlık durumunu, absorbe edilen düğüm sayılarını ve sivil veri filtreleme telemetrisini arayüze aktaran durum yönetim protokolü.

```
const DigiGuardDashboardBridge = {
    rpcGateway: "https://web4.digiguard.internal/rpc",
    initDashboardSession: async function(ghostIdKey) {
        console.log("[DASHBOARD_BRIDGE] Session initialized securely.");
    }
};
```

ADIM 6: KUANTUM ZAMAN DÜZENLEYİCİ VE REGÜLASYON MATRİSİ

Shor ve Grover algoritmalarının kuantum kaba kuvvet (brute-force) avantajlarını entropy havuzu ve zaman genleşmesi ile sıfırlayan katman.

```
class QuantumRegulationMatrix:
    def __init__(self):
        self.grover_entropy_bits = 256
        self.ether_time_dilation = 0.0001
```

ADIM 7: BARIŞÇİL İNSANLIK ANAYASASI VE KAPALI DEVRE UTILITY TOKEN HARCAMA ALANLARI

Sivil kullanıcıların ağa verdikleri güç karşılığında elde ettikleri tokenları harcayacakları otonom, borsa dışı alanları belirler.

```
class ConstitutionalUtilityRouter:
    def __init__(self):
        self.primary_constitution = "HUMANITARIAN_PROTECTION_CAN_MAL_ITIBAR_EMNIYETI"
        self.allowed_internal_services = ["ANTI_SLANDER_FILTER", "DEEP_SCAN_PHISHING_BLOCK"]
```

ADIM 8: POST-KUANTUM ÖZ-KANIT KATMANI (LATTICE-BASED SELF-PROVENANCE WATERMARK)

Ağın sahipsiz anomaliler üretmek yerine, devletlerin siber tarama radarlarına karşı paketi açmadan, onun zararsız ve insanlığa hizmet eden bir protokol olduğunu kanıtlayan kuantum dirençli filigran mührüdür.

```
class QuantumProtocolProvenance:
    def generate_quantum_passport(self, ghost_id, payload_hash):
        epoch_window = str(int(time.time() / 10))
        pre_image = f"{ghost_id}:{payload_hash}:{epoch_window}:DIGIGUARD_CONSTITUTION".encode()
        return hashlib.sha3_512(hashlib.sha3_512(pre_image).digest()).hexdigest()
```

ADIM 9: KÜRESEL KUANTUM HİJYEN VE BÜYÜK ENTEGRASYON KATMANI (GLOBAL ENTERPRISE SANITATION API)

Google, Microsoft, Apple gibi işletim sistemi devlerinin kuantum dalgalanması ve karmaşası anında çökmelerini engelleyen, DigiGuard'dan otonom uyumluluk sertifikası alarak kendi kernel katmanlarına gömebilecekleri açık entegrasyon kapısıdır.

```
class GlobalEnterpriseSanitationAPI:
    def issue_quantum_compliance_certificate(self, enterprise_name, public_infrastructure_hash):
        certificate_epoch = str(int(time.time() / 86400))
        raw_seal = f"{enterprise_name}:{public_infrastructure_hash}:{certificate_epoch}".encode()
        return hashlib.sha3_512(hashlib.sha3_512(raw_seal).digest()).hexdigest()
```

ADIM 10: ANTI-KUANTUM KARMAŞASI (ANTI-ENTANGLEMENT) SİMÜLASYON REGÜLATÖRÜ

Küresel internette meydana gelebilecek kuantum dalgalanmalarını sönümleyerek sivil cihazlardaki Daemon'ların aşırı işlem yükü altında kilitlenmesini veya donanımsal zarar görmesini engelleyen tampon katman.

```
class AntiEntanglementRegulator:
    def absorb_quantum_wave_shock(self, network_coherence_factor, matrix):
        if network_coherence_factor > 0.85:
            matrix.ether_time_dilation = 0.005
            return "ABSORBING_SHOCK_ACTIVE"
        return "STABLE"
```

ADIM 11: KURUMSAL ONBOARDING VE ALTYAPI KYC (INFRASTRUCTURE KYC) GATEWAY KONTRAKTI

Büyük dev şirketlerin (Google, AWS vb.) evrak yüklemek yerine kendi otonom altyapı bütünlüklerini, ASN numaralarını ve mülkiyetlerini ağa matematiksel olarak kanıtladığı insan müdahalesiz onboarding kapısıdır.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

contract DigiGuardEnterpriseGateway {
    struct EnterpriseProfile {
        string corporateName;
        uint256 registeredASN;
        bytes32 infrastructureHash;
        uint256 stakedDGD;
        string vipServiceTier;
        bool isSanitized;
    }

    mapping(address => EnterpriseProfile) public verifiedEnterprises;
    address public constitutionEnforcer;

    event EnterpriseOnboarded(address indexed corporateWallet, string name, string tier);
    event InfrastructureViabilityFailed(address indexed corporateWallet, string reason);

    constructor() { constitutionEnforcer = msg.sender; }

    function otonomInfrastructureKYC(
        string memory _name,
        uint256 _asn,
        bytes32 _infraHash,
        string memory _tier
    ) external payable {
        // Minimum VIP Staking Eşiği Kontrolü (Örn: 50,000 DGD Token Eşdeğeri)
        require(msg.value >= 50 ether, "ERR: MINIMUM_VIP_STAKE_NOT_MET");

        // Altyapı dürüstlük ve anayasaya uyumluluk taahhüdü doğrulaması
        require(_infraHash != bytes32(0), "ERR: INVALID_INFRASTRUCTURE_PROOF");

        verifiedEnterprises[msg.sender] = EnterpriseProfile({
            corporateName: _name,
            registeredASN: _asn,
            infrastructureHash: _infraHash,
            stakedDGD: msg.value,
            vipServiceTier: _tier,
            isSanitized: true
        });

        emit EnterpriseOnboarded(msg.sender, _name, _tier);
    }
}
```

ADIM 12: MESH-CRAWLING VE GEÇİCİ ADLİ RAPORLAMA (EPHEMERAL FORENSICS) MOTORU

Marka ve patent haklarını korumak için interneti sunuculara kaydetmeden, sivil düğümler üzerinden akan veri akışını yerinde (local) tarayan ve sıfır-bilgi kanıtlı rapor üreten otonom motor.

```python
class EphemeralForensicsEngine:
    def __init__(self):
        self.active_intellectual_property_hashes = {}

    def inject_enterprise_patent_dna(self, enterprise_address, patent_string):
        patent_hash = hashlib.sha3_256(patent_string.encode()).hexdigest()
        self.active_intellectual_property_hashes[patent_hash] = {
            "owner": enterprise_address,
            "live_violations_counter": 0
        }

    def local_mesh_packet_scan(self, packet_payload):
        # Sivil cihaz içinden akan paket taranır, ham veri asla diske yazılmaz
        payload_hash = hashlib.sha3_256(packet_payload.encode()).hexdigest()
        if payload_hash in self.active_intellectual_property_hashes:
            self.active_intellectual_property_hashes[payload_hash]["live_violations_counter"] += 1
            # Sapon Active Defense Engine tetiklenir ve rota izole edilir
            return {"action": "TRIGGER_SAPON_ISOLATION", "target_hash": payload_hash}
        return {"action": "PASS"}

    def generate_ephemeral_report(self, patent_hash):
        # Rapor çekildikten sonra bellekten tamamen yok edilen geçici log şablonu
        if patent_hash in self.active_intellectual_property_hashes:
            data = self.active_intellectual_property_hashes[patent_hash]
            report = {
                "patent_secure_hash": patent_hash,
                "total_prevented_violations": data["live_violations_counter"],
                "sapon_defense_status": "PROACTIVE_SHIELD_ACTIVE",
                "timestamp": time.time()
            }
            # Evrenden temizleme (Stateless İlkesi)
            data["live_violations_counter"] = 0
            return report
        return None
```

ADIM 13: KÜRESEL DAĞITIM VE ENTEGRASYON MANİFESTOSU

Tüm protokolün sunucusuz bir otonom yapı şeklinde küresel ağa dağıtılma hiyerarşisi ve validasyon kurallarıdır.

[VIP ENTERPRISE INTEGRATION VERDICT]
DigiGuard Fork Network, kurumsal devlerin sisteme dahil olurken bürokratik engellerle karşılaşmadığı, tamamen altyapı ve şifreleme gücü tabanlı bir otonom ekosistemdir. Büyük şirketler altyapılarını sivil ağ ile el sıkıştırarak (Handshake) internetin gelecekteki kuantum anomalilerine karşı en gelişmiş siber koruma kalkanını (Cerebron & Sapon) VIP düzeyde devreye alırlar.